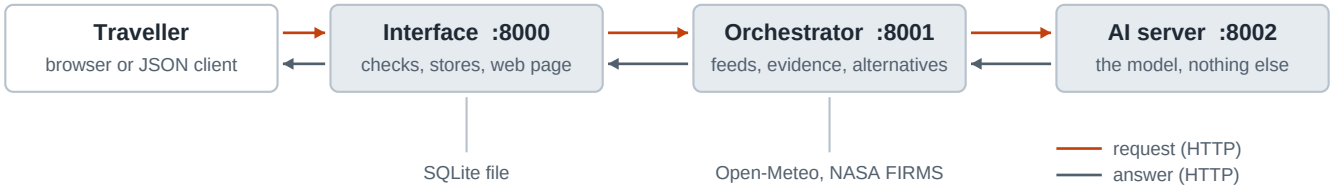
AshAway - Task 6: Distributed Architecture (preliminary report)

Software Engineering for CSAI 2026-2027 | Group 9 | Hammza Abas, Amin El Amrani, Rayan Fikri, Calvin Kakembo, Mikail Kaya, David Vasilescu | github.com/Davidtzuke/AshAway | code: src/ashaway/services/ | details: docs/architecture.md

What we built

AshAway now runs as three logical servers, each its own process and port, that talk only over HTTP. A traveller's request always goes interface → orchestrator → AI server and back. They run on one machine for now; each can move to another machine by changing one URL.



Server	Port	Does	Never does
Interface	8000	Web page and JSON API. Checks every request (place, dates, May-October, 14-day horizon) before anything else runs. Forwards valid ones to the orchestrator. Stores each request, its outcome and any feedback in SQLite, with nothing about the traveller.	Call the AI server. It has no address for it and imports no model code, XGBoost or pandas (tested).
Orchestrator	8001	Runs the Task 5 assessment logic unchanged: weather, fire and air feeds, features, evidence checks, alternatives, comparisons. Sends every prediction to the AI server through *RemoteModel*, which has the same methods as the local model.	Load the model.
AI server	8002	The only process that loads the model (models/live-v1). */model* gives version, features and risk cut-offs; */predict* gives calibrated probabilities and, on request, feature contributions. Refuses incomplete rows by name.	Know about places, dates or feeds.

How the servers talk, and why HTTP

Synchronous HTTP with JSON (FastAPI and uvicorn on the servers, httpx between them). A traveller waits for every answer, an assessment takes well under a second, and there are three services, so a message broker such as RabbitMQ would add a fourth process without changing what the traveller sees. If background jobs are ever needed, a queue goes into the orchestrator only. Each request gets an id that every server logs and returns (X-Request-ID), so one request can be followed through all three logs. All errors share one shape: *{error: {code, message, service}}*.

Failures. A bad request stops at the interface (400). An orchestrator or AI outage reaches the traveller as a 503 with no risk level. A failed feed is handled as in Task 5: the source is marked unavailable and 'insufficient evidence' replaces a low result. Every outcome is stored.

How to run it

`python -m ashaway.services` starts all three processes; open http://127.0.0.1:8000. `--replay` uses recorded 2023 weather, so a demo works without internet. Or start `ashaway-ai`, `ashaway-orchestrator` and `ashaway-interface` in three terminals.

Verification

ID	Level	Components	Checks	Passed
U9	Unit	RemoteModel with an in-memory fake AI server	Sends only the model's features, in order; reads probabilities; raises ModelUnavailable when unreachable or on HTTP 500	4/4
I7	Integration	AI server API with the real model	Same probabilities over HTTP as in-process; missing feature refused by name	8/8
I8	Integration	Orchestrator + AI server	Same assessment as Task 5; bad place refused before the AI server is contacted; AI down gives 503	6/6
I9	Integration	Interface + storage	Checks, forwards and stores; bad requests never forwarded; outages passed on as 503; feedback stored	13/13
I10	Integration	All three servers on real sockets	One request id end to end; identical answer to Task 5; stored; outages and feed failures reach the traveller	6/6

All **155 tests pass** (87 unit, 68 integration), including the 118 from Task 5, which run unchanged against the same logic. We also started the three servers as separate processes and sent Rhodes, 18-24 July 2023, through the web page: the answer was HIGH, it made 13 calls to the AI server in about 0.26 s, and the same request id appeared in all three logs.

Still preliminary

The servers trust each other: across a network they would need HTTPS and a shared key. SQLite suits one interface server. Requests are synchronous, and with live feeds an assessment with alternatives takes a few seconds. Not yet done: running the servers on separate machines or in the cloud, a FIRMS key for live fire detections (T12), and the full web design (T14).